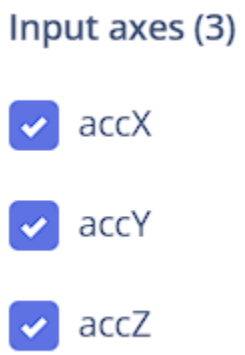
Section 1: Edge Impulse Results

Public Edge Impulse project link:
https://studio.edgeimpulse.com/public/995728/live

1. Raw inputs: State the raw inputs used in the Edge Impulse project.



The sampling frequency was 100 Hz with a 2000 ms window size and 220 ms stride.

2. NN classifier features: State the type of features passed into the NN classifier, the number of input features, and at least five actual feature names used to train the classifier.

We used spectral features, 33 input features, these were the 5 most important features:

- accX RMS
- accX Peak 1 Height
- accY Peak 3 Height
- accX Peak 1 Freq
- accY RMS

3. NN classifier outputs: State the outputs of the NN classifier.

Nominal and off

4. Anomaly detector features: State the type of features passed into the anomaly detector, the number of features used, and the feature names used to train the anomaly detector.

Feature type: spectral analysis features
Number of features used: 4
Feature names:

- accX RMS
- accX Peak 1 Height

- accY Spectral Power 2.0 - 5.0 Hz
- accZ Spectral Power 2.0 - 5.0 Hz

5. Deployment evidence: Include a screenshot showing the Edge Impulse model running on the Arduino Nano33 BLE Sense and producing inference results.

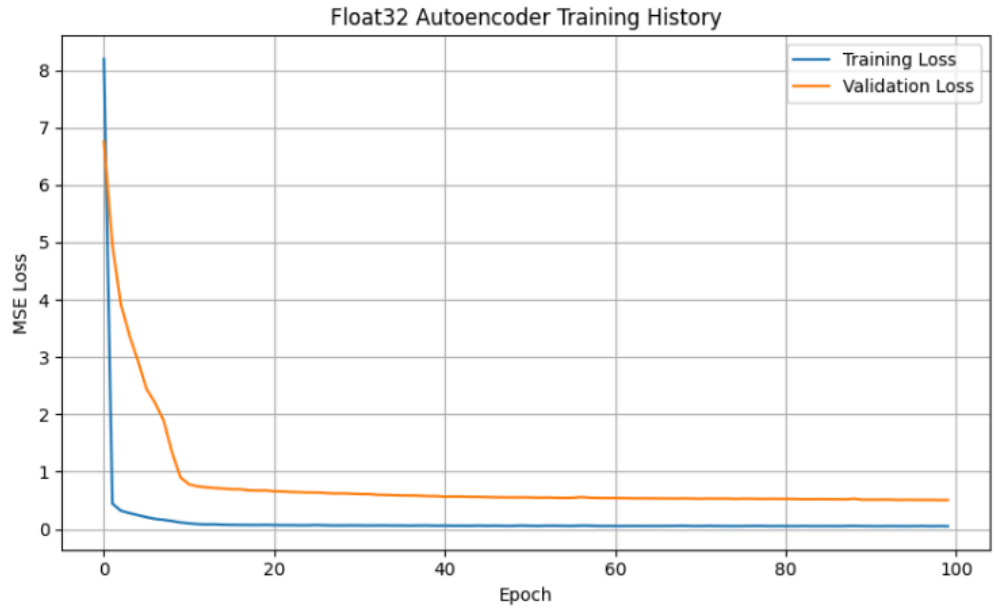
```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 611 us, anomaly 527 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.022402
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 610 us, anomaly 509 us, postprocessing 50 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.022552
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 44 ms, inference 611 us, anomaly 6 ms, postprocessing 49 us
#Classification predictions:
  nominal: 0.437500
  off: 0.562500
Anomaly prediction: 16.023378
```

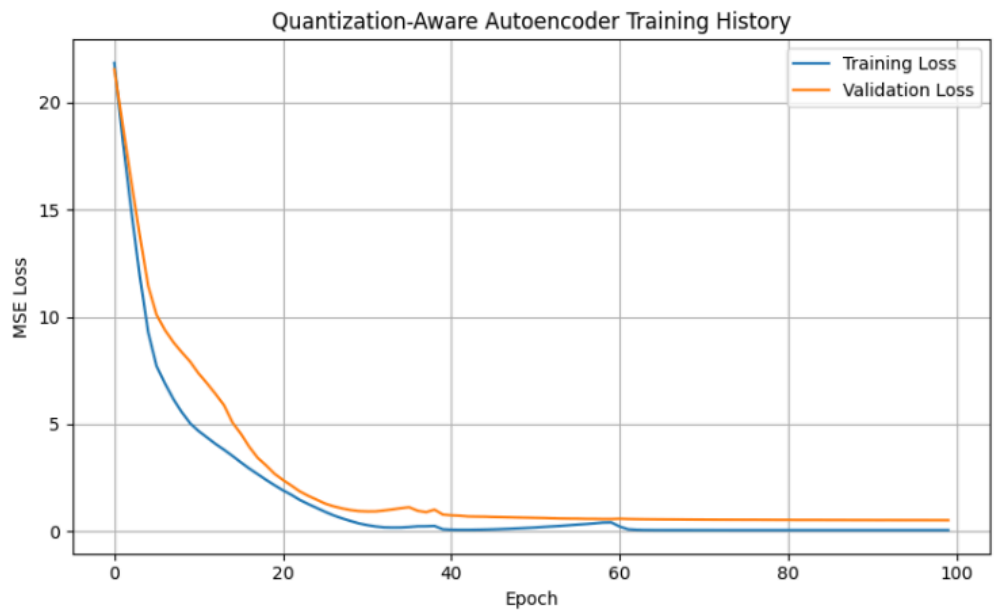
6. Short interpretation: Briefly interpret the observed deployment results. Discuss whether the classifier output should always be trusted and under what conditions it may or may not be reliable.

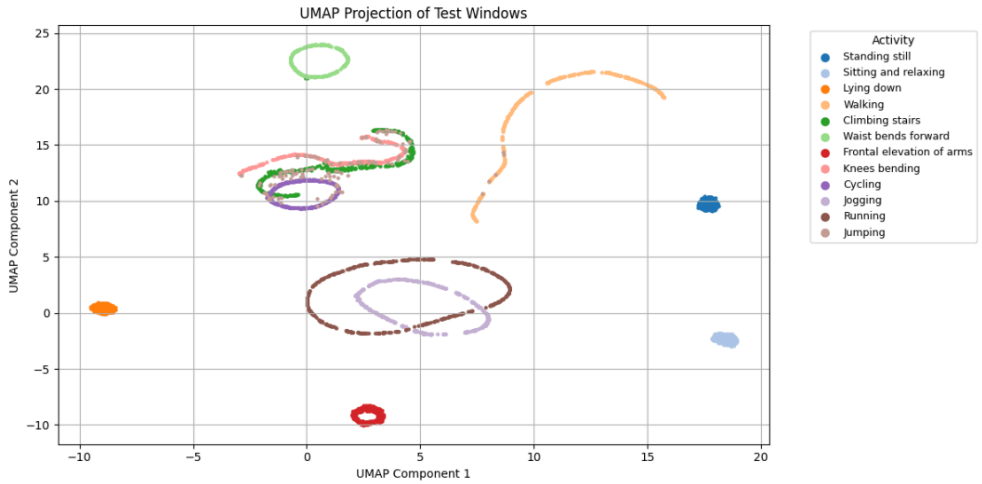
During deployment on the Arduino Nano 33 BLE Sense, the model was able to successfully run real-time inference and produce both classification outputs and anomaly scores. The classifier consistently predicted the “off” class, but only with around 56% confidence, while “nominal” still remained fairly high at around 44%. This screenshot is when the ble was not moving at all.

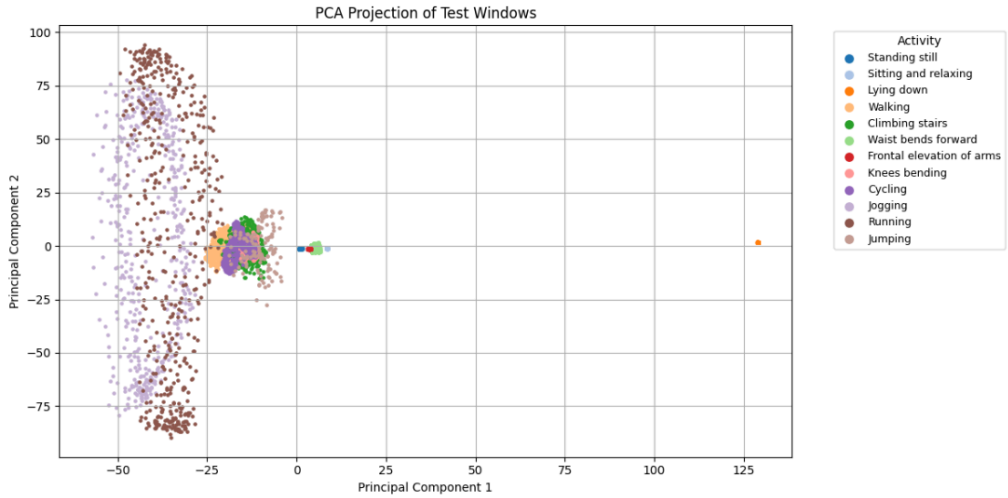
One interesting thing is that the model achieved 100% validation accuracy during training, but the live deployment results were much less confident. This shows that classifier outputs should not always be blindly trusted, especially with limited or less varied training data. Changes in motion, sensor orientation, noise, or real-world conditions can all affect reliability. The relatively high anomaly scores also suggest that the live input data may not closely match the data the model was originally trained on. Perhaps the model was overfit or the real world does not exactly match the dataset it was trained on.

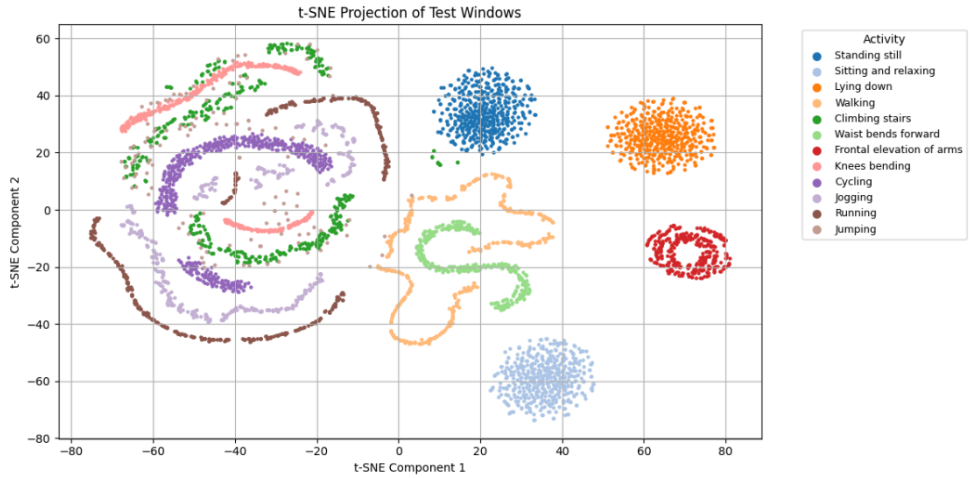
Section 2: Autoencoder Model Results

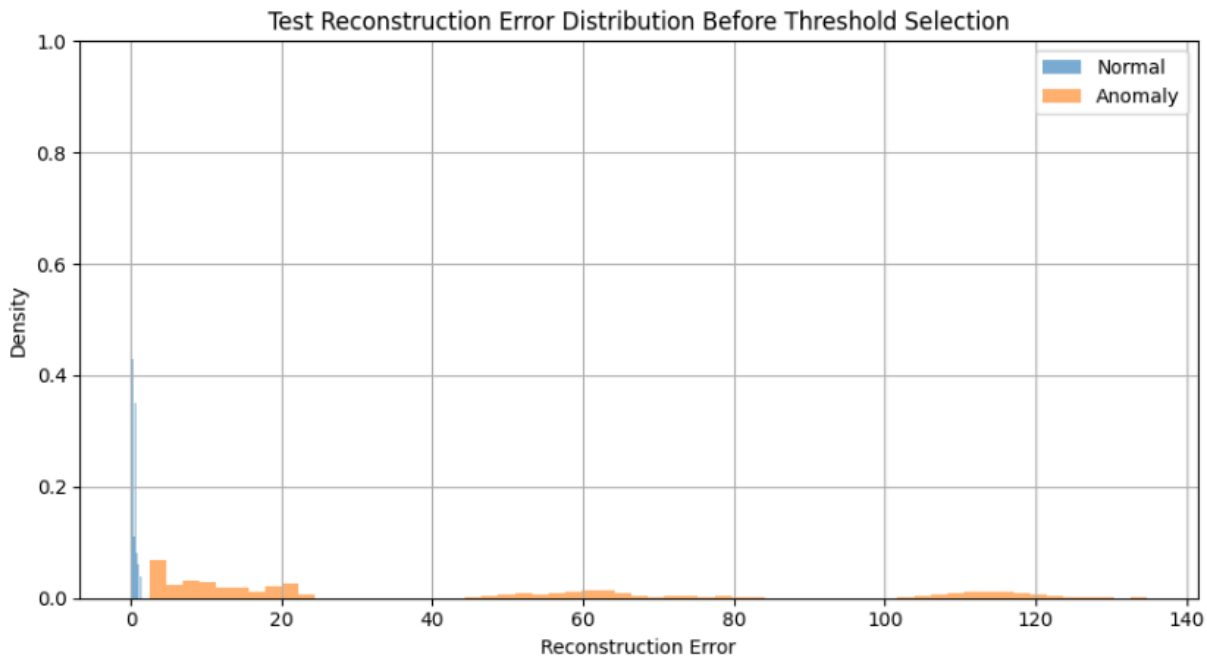


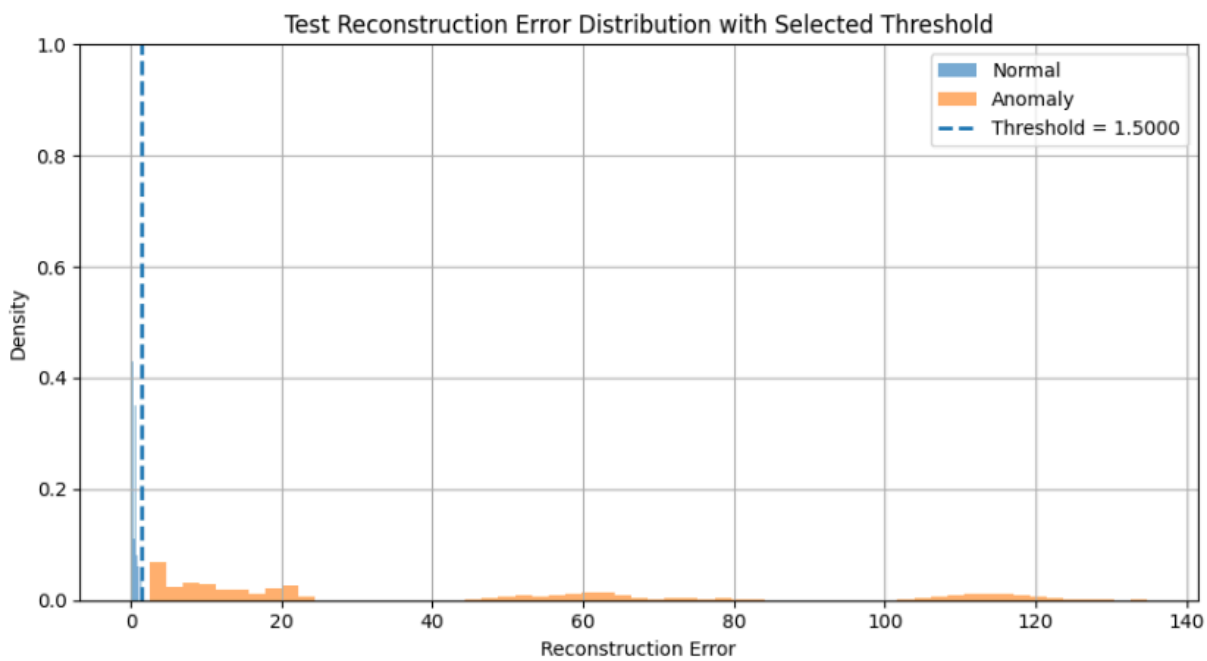



UMAP Projection of Test Windows
UMAP Component 1
UMAP Component 2
Activity
Standing still
Sitting and relaxing
Lying down
Walking
Climbing stairs
Waist bends forward
Frontal elevation of arms
Knees bending
Cycling
Jogging
Running
Jumping

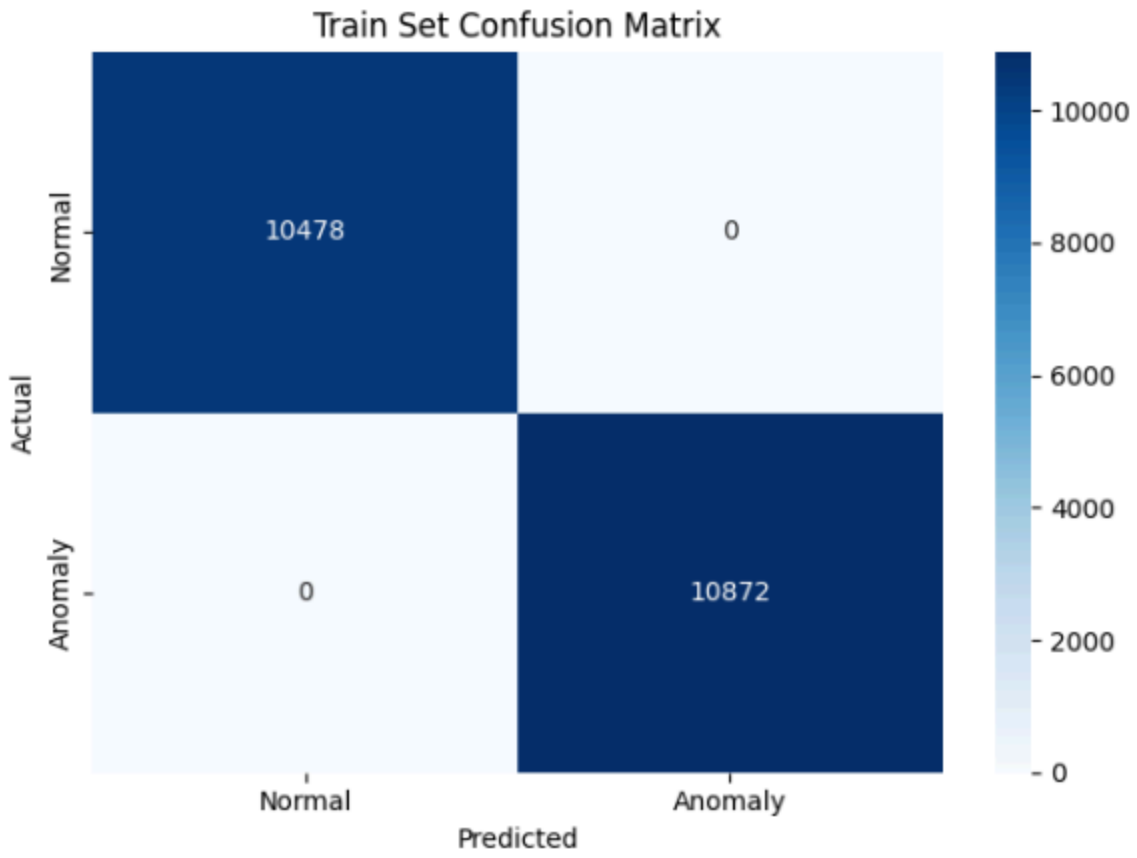

PCA Projection of Test Windows
Principal Component 1
Principal Component 2
Activity
Standing still
Sitting and relaxing
Lying down
Walking
Climbing stairs
Waist bends forward
Frontal elevation of arms
Knees bending
Cycling
Jogging
Running
Jumping

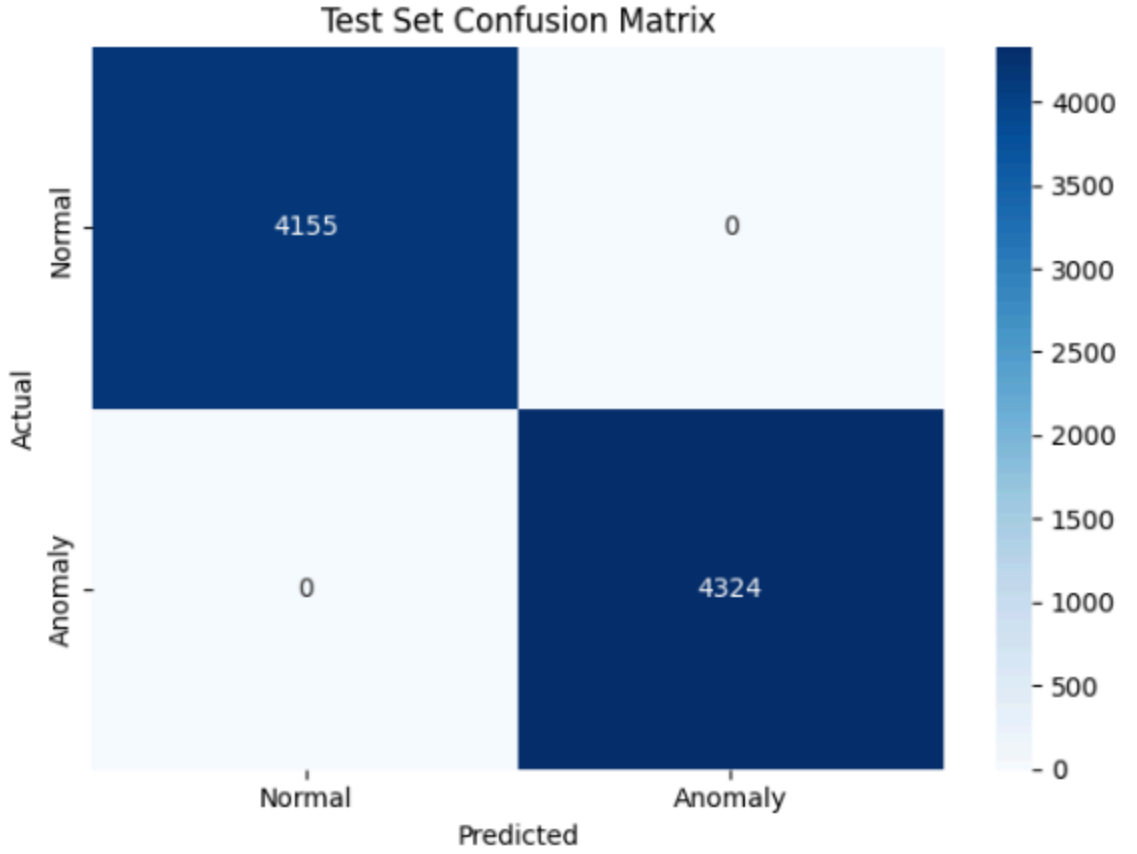

t-SNE Projection of Test Windows
t-SNE Component 1
t-SNE Component 2
Activity
Standing still
Sitting and relaxing
Lying down
Walking
Climbing stairs
Waist bends forward
Frontal elevation of arms
Knees bending
Cycling
Jogging
Running
Jumping



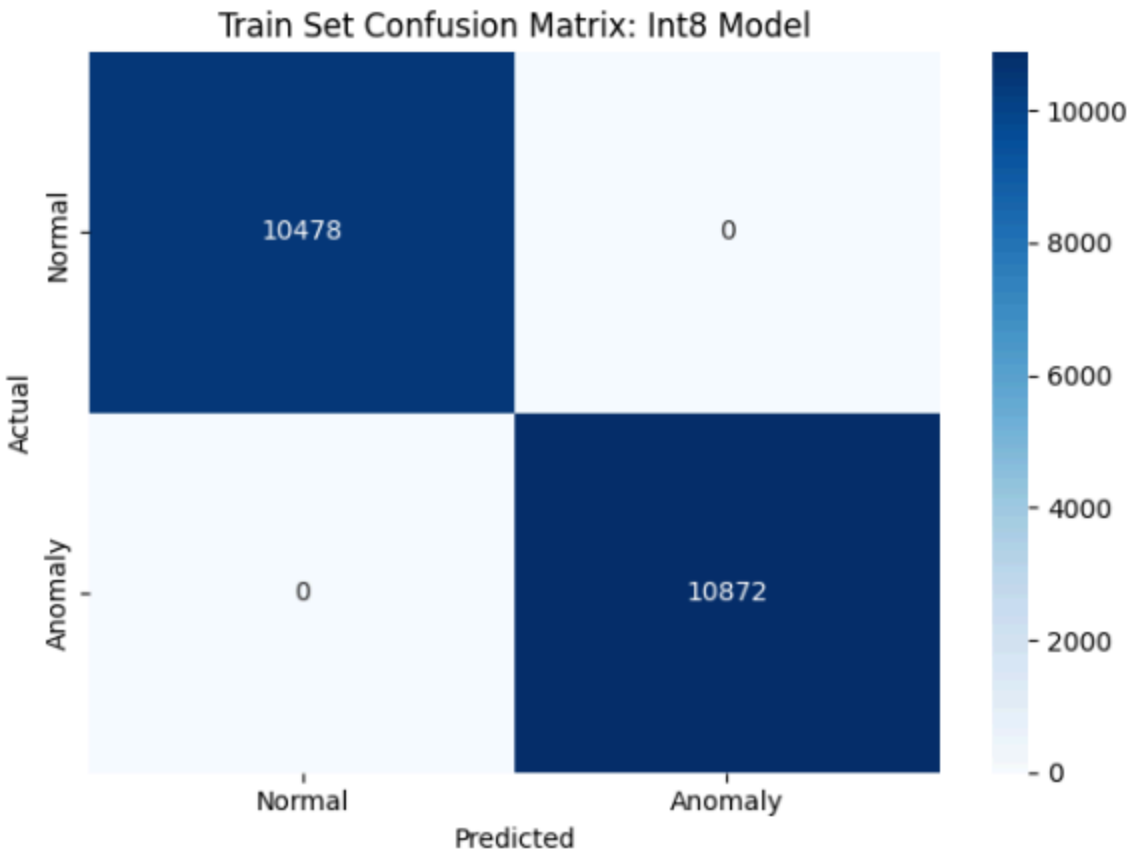


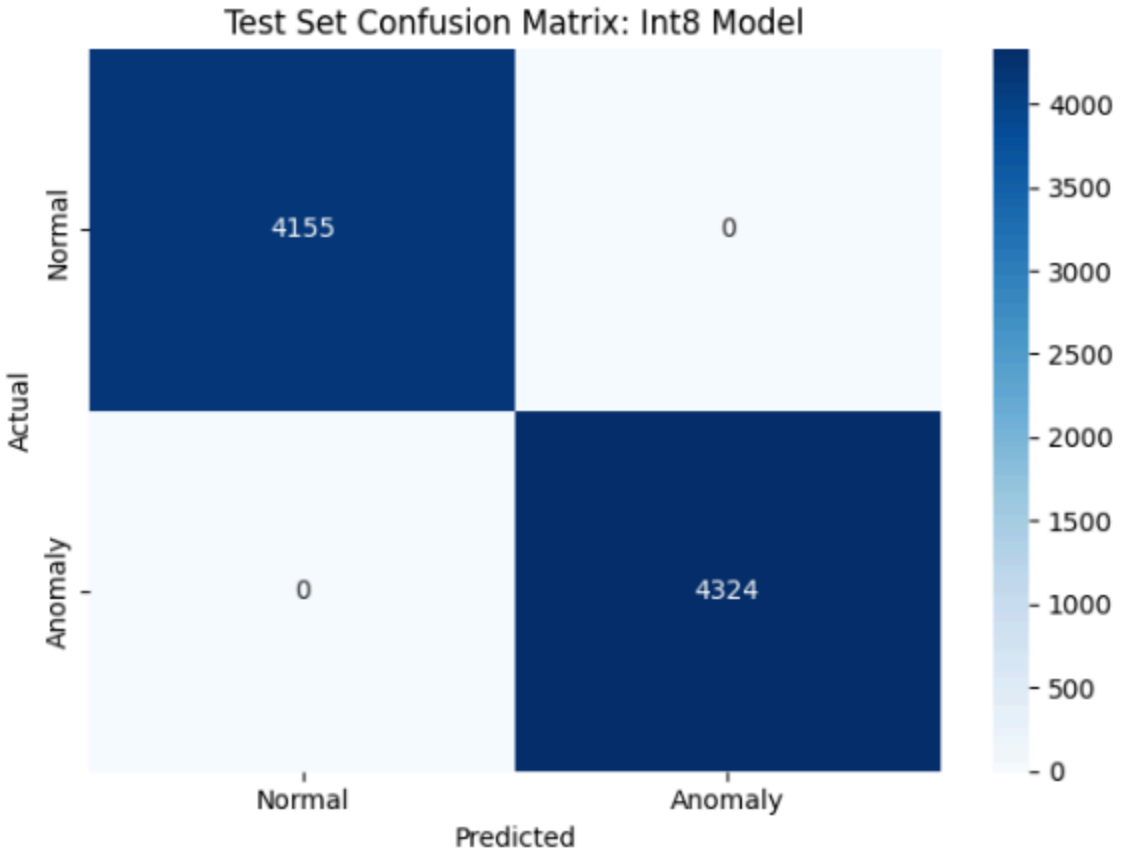
Threshold = 1.5


Train Set Confusion Matrix
10478
0
0
10872
Normal
Anomaly
Actual
Normal
Anomaly
Predicted
10000
8000
6000
4000
2000
0


Test Set Confusion Matrix
4155
0
0
4324
Normal
Anomaly
Actual
Normal
Anomaly
Predicted
4000
3500
3000
2500
2000
1500
1000
500
0

```
Train set evaluation
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00     10478
 Anomaly (1)       1.00      1.00      1.00     10872

    accuracy                           1.00     21350
   macro avg       1.00      1.00      1.00     21350
weighted avg       1.00      1.00      1.00     21350

Test set evaluation
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00      4155
 Anomaly (1)       1.00      1.00      1.00      4324

    accuracy                           1.00      8479
   macro avg       1.00      1.00      1.00      8479
weighted avg       1.00      1.00      1.00      8479
```


Train Set Confusion Matrix: Int8 Model
10478
0
0
10872
Normal
Anomaly
Actual
Normal
Anomaly
Predicted
10000
8000
6000
4000
2000
0


Test Set Confusion Matrix: Int8 Model
4155
0
0
4324
Normal
Anomaly
Actual
Normal
Anomaly
Predicted
4000
3500
3000
2500
2000
1500
1000
500
0

```
Train set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00     10478
 Anomaly (1)       1.00      1.00      1.00     10872

    accuracy                           1.00     21350
   macro avg       1.00      1.00      1.00     21350
weighted avg       1.00      1.00      1.00     21350

Test set evaluation: int8 model
              precision    recall  f1-score   support

  Normal (0)       1.00      1.00      1.00      4155
 Anomaly (1)       1.00      1.00      1.00      4324

    accuracy                           1.00      8479
   macro avg       1.00      1.00      1.00      8479
weighted avg       1.00      1.00      1.00      8479
```

```
Result: Anomaly detected
Reconstruction error: 1.625174
Result: Anomaly detected
Reconstruction error: 1.443181
Result: Normal activity
Reconstruction error: 1.304863
Result: Normal activity
```

Section 3: Discussion Questions

What threshold did you choose for anomaly detection, and why?

First I tried to be cool by calculating it by doing:
threshold = np.mean(train_normal_errors) + 3 * np.std(train_normal_errors)
print("Selected threshold:", threshold) # this printed 0.6124692484736443

But after looking at the graph and confusion matrices I manually adjusted it and ended up with a value of 1.5, which also matched what was already in the arduino sketch so that was nice.

How does reconstruction error help distinguish normal and anomalous activity?

Since the encoder was trained on mostly normal data, it learned how to reconstruct those patterns pretty accurately so when normal activity is passed to the model its reconstruction error is small. However when the motion is unusual or not something it's seen before, it has not learned the patterns to reconstruct the motion very well so the reconstruction error is large. By comparing this to our set threshold we can classify if the motion is normal or anomalous.

What information from the Python notebook must be preserved when deploying the model to Arduino?

The trained model g_model byte array and g_model_len size value. Additionally some of the same preprocessing assumptions already given to us by the .ino included with the lab like sampling rate and shape and stuff. Also the error threshold.

Why is it important for the Arduino sketch to use the same preprocessing assumptions as the notebook?

The training data was processed in a very specific way and if the arduino code processes it differently then the model may not behave correctly. If the data is scaled, normalized, windowed, or ordered differently, or if the sample rate is different it may cause reconstruction errors to be very off, causing the model to behave worse even if it is correct.

What are the main limitations of this anomaly detection method when deployed on a microcontroller?

The model can only detect anomalies relative to the data it was trained on. If it is limited or there is a different kind of movement like rowing or skydiving or something like that, it may detect anomalies even when there are none. Additionally, since the threshold picking is somewhat subjective, since my statistical choice wasn’t the best one, one person may choose a threshold that detects too many anomalies while another may choose one that ignores some. Additionally, due to the limitations of hardware like RAM and storage more advanced models may fail to run at all or be very effective at this kind of detection. Also sensor noise or differences between users.